

SPECIFIKACIJA PROJEKTA

Softverski agenti - 2026

Student: Luka Kešelj | RA 102/2022 | maj 2026.

Naziv: Pametna kuca - Upravljanje IoT uredajima pomou federativnog učenja i akorskog modela

Planirana ocena: 9 - FedAvg federativno učenje + genericki akorski radni okvir (Go) sa svim obaveznim elementima i dvije dodatne funkcionalnosti: **Supervizija akora** i **Perzistencija stanja akora**.

1. Opis problema

Sistem simulira mrezu IoT senzora u pametnoj kuci koji prikupljaju podatke o stanju prostorija (temperatura, vlaznost, osvetljenje, prisustvo osoba). Svaki senzor poseduje lokalni model koji na osnovu sopstvenih merenja predvoda optimalne uslove u prostoriji. Kroz federativno učenje, senzori periodicno dele parametre modela sa centralnim koordinatorom - bez slanja sirovih podataka, cime se cuva privatnost.

Akorski sistem, implementiran u Go uz gRPC transport, upravlja asinhronom komunikacijom i donosi odluke o aktivaciji uredaja (klima, grejanje, rolete, rasveta) na osnovu globalnog modela. Svaki aktor je Go goroutina sa sopstvenim kanalom (mailbox-om) i jasno definisanim lifecycle-om.

2. Federativno učenje

2.1 Algoritam - FedAvg

Implementira se **Federated Averaging (FedAvg)** algoritam (McMahan et al., 2017). Svaki cvor lokalno trenira model (MLP) gradijentnim spustom na sopstvenim podacima, bez eksternih ML biblioteka (implementacija u cistom Go-u). Koordinator vrši ponderisano usrednjavanje tezina klijenata i vraca azurirani globalni model.

Koraci FedAvg algoritma:

- Koordinator salje inicijalne parametre globalnog modela svim SensorActorima.
- Svaki SensorActor trenira lokalni model kroz E epoha (lokalni SGD).
- SensorActori salju azurirane tezine CoordinatorActor-u.
- Koordinator agregira: $w_{global} = \sum(n_k / N) \times w_k$
- Ponavljja se kroz R rundi dok model ne konvergira.

2.2 Skup podataka

Atribut	Opseg vrednosti	Jedinica
Temperatura	15 - 35	C

Atribut	Opseg vrednosti	Jedinica
Vlaznost	20 - 80	%
Osvetljenje	50 - 1000	lx
Prisustvo osoba	0 / 1	boolean
Doba dana	0 - 23	h

Svatom senzoru dodeljuje se lokalni skup od ~800 uzoraka. Podržana je IID i non-IID distribucija podataka po prostorijama.

2.3 Distribucija treniranja

Svaki SensorActor nezavisno i asinhrono trenira model u pozadinskoj goroutini. Po završetku runde, šalje parametre CoordinatorActor-u putem gRPC poziva. Koordinator sakuplja parametre, agregira ih FedAvg algoritmom i propagira globalni model nazad svim aktorima.

2.4 Evaluacija rezultata

Metrika	Opis
MSE / RMSE	Osnovna metrika greske predikcije uslova u prostoriji
MAE	Srednja apsolutna greska - lako interpretabilna
R2 skor	Koeficijent determinacije - kvalitet prilagodavanja modela
Konvergencija	Grafikon gubitka po rundi (globalni vs. lokalni modeli)

Za prezentaciju bice prikazani grafici greske pre i posle svake runde federativne agregacije, kao i konvergencija modela kroz runde (terminalni ispis tabele ili SVG izlaz iz Go-a).

3. Aktori i komunikacija

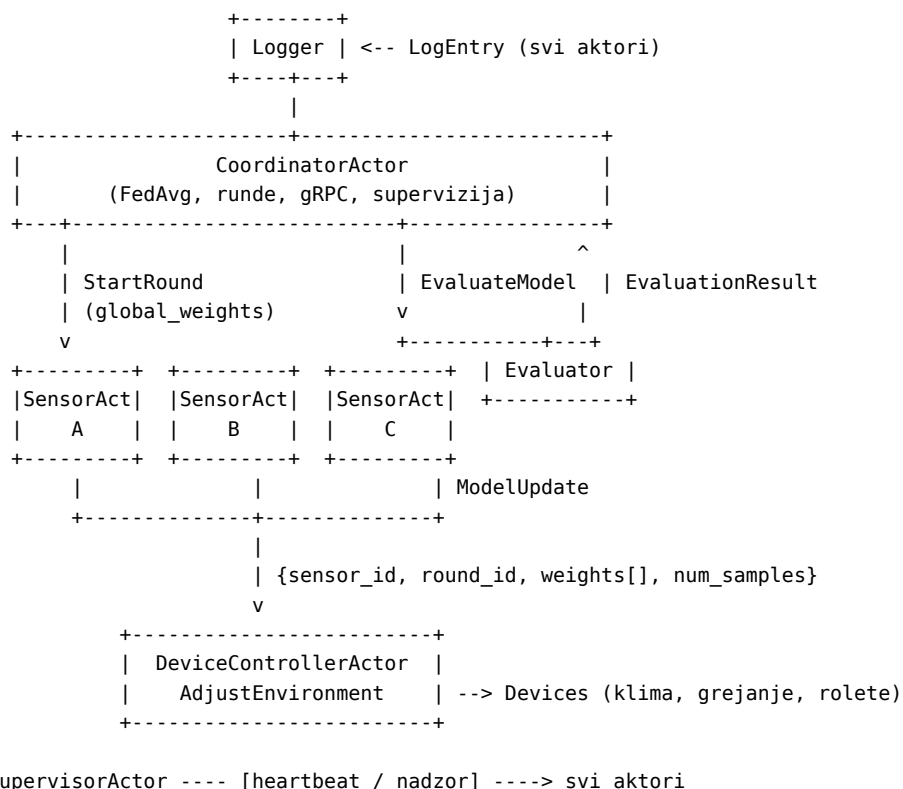
3.1 Vrste aktora i uloge

Aktor	Uloga	Poruke
SensorActor	Generise podatke prostorije, trenira lokalni ML model	StartTraining, ModelUpdate, GlobalModelUpdate
CoordinatorActor	Upravlja rundama FL-a, agregira modele (FedAvg), distribuira globalni model	ModelUpdate, GlobalModelUpdate, RequestStatus, RoundComplete
DeviceControllerActor	Simulira IoT uredaje: klima, grejanje, rolete, svetlo	AdjustEnvironment, DeviceStatus
LoggerActor	Belezi metrike, logove, MSE/RMSE po rundi	LogMetrics, LogEvent
SupervisorActor	Nadzire sve aktore, detektuje greske, restartuje pale aktore	ActorFailed, RestartActor, StatusCheck

3.2 Poruke izmedju aktora

Poruka	Posiljalac -> Primalac	Sadržaj	Namena
StartTraining	Coordinator -> Sensor*	round_id, model_weights[], learning_rate, epochs	Pokretanje runde lokalnog treniranja
ModelUpdate	Sensor -> Coordinator	sensor_id, round_id, weights[], num_samples, loss	Slanje lokalnih azuriranja modela
GlobalModelUpdate	Coordinator -> Sensor*	round_id, global_weights[], global_loss	Distribucija agregiranog globalnog modela
RoundComplete	Coordinator -> Logger	round_id, global_loss, num_clients, elapsed_ms	Izvestaj o završenoj rundi
AdjustEnvironment	Coordinator -> DeviceCtrl	room_id, target_temp, humidity, light_level	Aktivacija IoT uređaja
DeviceStatus	DeviceCtrl -> Logger	room_id, device_type, action, timestamp	Status izvršene akcije
ActorFailed	Runtime -> Supervisor	actor_id, actor_type, error_msg, restart_count	Notifikacija o padu aktora
RestartActor	Supervisor -> Runtime	actor_id, strategy, delay_ms	Nalog za restart neuspješnog aktora
LogMetrics	Any -> Logger	round_id, mse, rmse, mae, r2_score	Zapis evaluacionih metrika
StatusCheck	Supervisor -> Any	actor_id	Provjera živosti aktora

3.3 Dijagram komunikacije aktora



4. Implementacija akterskog radnog okvira

Radni okvir je **genericki** i nezavisan od logike federativnog ucenja. Implementiran je u **Go 1.22+** koristeći goroutine i kanale kao prirodnu osnovu za akterski model.

4.1 Obavezni elementi

Element	Opis implementacije
Aktori	Svaki aktor je Go goroutine sa jedinstvenim ActorID (UUID). Interface Actor definise metode Receive(msg Message), OnStart(ctx) i OnStop().
Asinhrono slanje/primanje	Poruke se salju neblokirajuci (ActorRef.Tell(msg)). Citanje iz mailboxa je asinhrono u pozadinskoj goroutini aktora.
Sanduce (Mailbox)	Baferisani Go kanal (chan Message) po aktoru. Podržani bounded i unbounded mailbox sa FIFO strategijom.
Promena ponasanja	context.Become(newBehavior BehaviorFunc) - aktor zamenjuje Receive funkciju u toku izvršavanja. Stanja: Idle -> Training -> SendingResults -> Idle.
Lifecycle događaji	Sistemske poruke: Started, Stopping, Stopped, Restarting. Hook-ovi: OnPreStart, OnPostStop, OnPreRestart.
Udaljena komunikacija	gRPC transport sloj. RemoteActorRef transparentno prosljeđuje poruke udaljenom aktoru (Protocol Buffers). Podržani Tell i Ask (sa timeout-om).

4.2 Dodatne funkcionalnosti

4.2.1 Supervizija aktora

SupervisorActor nadgleda stanje svih aktora putem periodičnih heartbeat provjera (StatusCheck svakih 5s). U slučaju pada, supervizor automatski restartuje pali aktor, obnavlja persistirano stanje i beleži događaj u LoggerActor.

Strategija restarta	Opis
OneForOne	Restartuje samo neuspjesni aktor - ostali nastavljaju normalno
ExponentialBackoff	Progresivno odlaganje restarta (1s -> 2s -> 4s -> max 30s)
Escalation	Ako aktor premasi max_restarts u window vremenu, greska se eskalira roditeljskom supervizoru

Demonstracija: simulacija pada SensorActor-a tokom aktivne runde treniranja - supervizor otkriva pad, restartuje aktor i obnavlja model iz trajnog stanja.

4.2.2 Perzistencija stanja aktora

Svaki aktor periodično serijalizuje interno stanje (parametri modela, brojaci rundi, status uređaja, zadnji globalni model) na disk u JSON formatu. Pri ponovnom pokretanju, aktori učitavaju poslednje sacuvano stanje i nastavljaju od tacke prekida.

Aktor	Sta se persistira	Format
SensorActor	Lokalne tezine modela, round_id, num_samples	JSON
CoordinatorActor	Globalne tezine, round_id, lista aktivnih klijenata	JSON
DeviceControllerActor	Status svakog uredaja, zadnja aktivacija	JSON
LoggerActor	Metrike po rundi (MSE, RMSE, MAE, R2)	JSON (append)

5. Detalji implementacije

Komponenta	Tehnologija	Svrha
Programski jezik	Go 1.22+	Nativna podrška za goroutines i channels
Aktorski sistem	Vlastita impl. (goroutine + chan)	Genericki aktorski radni okvir nezavisan od FL logike
Federativno ucenje	Vlastita impl. MLP + FedAvg u Go-u	Bez eksternih ML biblioteka - cisti Go math
Udaljena komunikacija	gRPC + Protocol Buffers	Efikasna binarna serijalizacija, bidirekciono streaming
Perzistencija stanja	JSON (encoding/json)	Serijalizacija stanja aktora na disk
Supervizija	Vlastiti SupervisorActor	Hijerarhija supervizora, strategije restarta
Konfiguracija	YAML + env varijable	Broj senzora, rundi, learning rate, putanje
Logovanje	Go log/slog	Metrike treniranja i performanse sistema
Testiranje	Go testing + Testify	Unit testovi framework-a, integracioni testovi
Kontejnerizacija	Docker + Docker Compose	Simulacija vise masina lokalno; deploy na stvarnim masinama

6. Struktura repozitorijuma

```

/
|-- actor-framework/ <- Genericki aktorski radni okvir
| |-- actor.go <- Actor interface i ActorRef
| |-- mailbox.go <- Mailbox (chan Message) implementacija
| |-- context.go <- ActorContext, Become/Unbecome
| |-- system.go <- ActorSystem (lifecycle management)
| |-- remote/ <- gRPC transport sloj + .proto definicije
| `-- supervision/ <- SupervisorActor, strategije restarta
|
|-- smart-home/ <- FL aplikacija (koristi actor-framework)
| |-- actors/ <- SensorActor, CoordinatorActor, itd.
| |-- model/ <- MLP implementacija, FedAvg agregacija
| |-- data/ <- Generisanje i particija IoT dataseta
| |-- persistence/ <- JSON serijalizacija stanja aktora
| `-- evaluation/ <- MSE, RMSE, MAE, R2 metrike
|
|-- demo/ <- Demo skripte i konfiguracije
|-- docker-compose.yml

```

7. Demonstracija funkcionalnosti

#	Scenario demonstracije
1	Lokalna demonstracija (Docker Compose): pokretanje 3 SensorActora i CoordinatorActora u izolovanim kontejnerima na jednoj masini
2	Visemasinska demonstracija: SensorActori na jednoj masini komuniciraju sa CoordinatorActor-om na drugoj putem gRPC veze
3	Supervizija: simulacija pada SensorActor-a tokom treniranja - automatski restart i obnavljanje stanja iz JSON fajla
4	Perzistencija: gasenje i ponovan start sistema - aktori nastavljaju od zadnje sacuvane tacke bez ponovnog treniranja
5	Konvergencija modela: ispis tabele MSE/RMSE po rundi i vizualizacija pada greske kroz federalne runde

8. Planirana ocena

Kriterijum	Status
Federativno ucenje (FedAvg, MLP u Go-u)	OK
Aktorski okvir - obavezni elementi (Go goroutine + gRPC)	OK
Supervizija aktora (dodatna funk. 1)	OK
Perzistencija stanja aktora (dodatna funk. 2)	OK
Demonstracija na 2 masine/procesa	OK
Testovi (Go testing + Testify)	OK